

High-Level Design

RAG Customer Support Assistant

1. Problem & Scope

We need a chatbot that answers customer questions from a PDF knowledge base. It should retrieve relevant content, generate answers with an LLM, and escalate to a human agent when it cannot answer confidently.

What is in scope

- Single PDF knowledge base
- CLI interface for the user
- Embedding + vector search using ChromaDB
- Answer generation with GPT-3.5 or a local LLM
- Conditional routing: auto-answer vs. human escalation (HITL)

2. Architecture Diagram

Data flows in two phases: ingestion (offline) and query handling (online).

INGESTION (run once)

PDF File → Load Pages → Split into Chunks → Embed → Store in ChromaDB

QUERY (every user message)

User Question



[input_node] → [retrieve_node] → [route_node]

(ChromaDB)



[llm_node] [hitl_node]



[output_node] → Answer to User

3. Components

Document Loader

Reads the PDF using LangChain PyPDFLoader. Outputs one Document per page.

Chunker

Splits pages into 500-character chunks with 50-char overlap using RecursiveCharacterTextSplitter.

Embedding Model

Converts text to vectors using sentence-transformers/all-MiniLM-L6-v2 (free, runs locally).

Vector Store

ChromaDB stores and searches embeddings. Persisted to disk; no server required.

Retriever

Runs similarity search on ChromaDB and returns the top-3 most relevant chunks.

LLM

GPT-3.5-turbo (or Ollama locally) reads the retrieved chunks and writes the answer.

Graph Engine

LangGraph StateGraph connects all nodes and handles conditional routing.

HITL Module

If no relevant chunks are found, execution pauses and a human agent types the answer.

4. Data Flow

Ingestion

- Load PDF → list of page Documents
- Split Documents → list of 500-char chunks
- Embed each chunk → 384-dim vector
- Store all vectors in ChromaDB collection 'support_kb'

Query lifecycle

- User types a question
- Question is embedded with the same model
- ChromaDB returns top-3 similar chunks
- Router checks: chunks found AND no 'human' keyword?
- Yes → build prompt with chunks → LLM → print answer
- No → ask human agent to type answer → print answer

5. Technology Choices

ChromaDB

Local, file-based, no setup needed. Perfect for a single-machine project.

LangGraph

Makes the workflow explicit as a graph. Easy to add or change nodes later.

sentence-transformers

Free local embeddings. No API key required.

GPT-3.5-turbo

Fast and affordable. Can be swapped with Ollama for a fully local setup.

6. Scalability Considerations

- Large documents: ChromaDB handles millions of vectors; chunk size keeps each request fast.
- High query load: wrap main.py with FastAPI; each request is stateless and can scale horizontally.
- Latency: cache frequent queries; reduce top-k from 3 to 1 for speed.
- Multiple PDFs: store document name as chunk metadata; filter by it at retrieval time.